Lab: Building a Complete RESTful API (CRUD Operations)

Overall Objectives:

- Apply REST principles to build a complete set of CRUD (Create, Read, Update, Delete) endpoints for a single resource.
- Implement `PUT` and `DELETE` operations in FastAPI.
- Reinforce the use of Pydantic models for request bodies and responses.
- Practice testing a full API lifecycle using both the interactive `/docs` and `curl`.

Your Choice of API

For this lab, you have two options. Both will teach you the same concepts. Choose the one that interests you more!

- Option A: Enhance the Questions API
  - Pros: You'll build upon the code you've already written, reinforcing what you did yesterday.
  - Focus: Adding the missing `PUT` and `DELETE` functionality to a familiar project.
- Option B: Build a New IoT Device API
  - Pros: You'll get to practice building a complete API from a blank file, which is great for solidifying your understanding of the setup process. The theme is highly relevant to IoT.
  - Focus: Creating a full CRUD API from scratch.

Option A: Enhance the Questions API

Part 1: Setup

1. Open Your Project:
   - Open your `main_api.py` file from the previous lab. It should already contain:
     - Pydantic models: `QuestionBase`, `QuestionCreate`, `Question`.
     - A `fake_db` list.
     - Endpoints for `POST /questions/`, `GET /questions/`, and `GET /questions/{question_id}`.
2. Run Your Server:
   - Open the integrated terminal and start the Uvicorn server so you can test as you go:

```
uvicorn main_api:app --reload
```

Part 2: Implement the `DELETE` Endpoint

1. The Goal: Create an endpoint that removes a question from our `fake_db` based on its `id`.
2. Add the Code:
   - In `main_api.py`, add the following new path operation:

```
# Make sure HTTPException is imported from fastapi
from fastapi import FastAPI, HTTPException
# ... your other code ...

@app.delete("/questions/{question_id}", response_model=Question)
async def delete_question(question_id: int):
    # Find the question to delete
    question_to_delete = None
    for question in fake_db:
        if question["id"] == question_id:
            question_to_delete = question
            break

    if question_to_delete is None:
        raise HTTPException(status_code=404, detail="Question not
found")

    # Remove the question from the list
    fake_db.remove(question_to_delete)

    # Return the item that was deleted
    return question_to_delete
```

3. Test It:
   - Using `/docs`: Refresh the documentation page. Find the new `DELETE` endpoint. Try deleting an existing question (e.g., with `id` 2). You should get a `200 OK` response with the deleted question's data. Then, try deleting it again; you should get a `404 Not Found`.
   - Using `curl`:

```
# First, see what's there
curl http://127.0.0.1:8000/questions/ | jq

# Delete item with id 2
curl -X DELETE http://127.0.0.1:8000/questions/2 | jq
```

```
# Verify it's gone
curl http://127.0.0.1:8000/questions/ | jq
```

Part 3: Implement the `PUT` Endpoint (Full Update)

1. The Goal: Create an endpoint that replaces the data for an existing question.
2. Create the Update Model:
   - A `PUT` request should replace the resource. We'll use our `QuestionCreate` model, which contains all the fields a user can modify (`question` and `answer`).
3. Add the Code:
   - In `main_api.py`, add this new path operation:

```
@app.put("/questions/{question_id}", response_model=Question)
async def update_question(question_id: int, question_update:
QuestionCreate):
    # Find the index of the question to update
    question_index = -1
    for index, q in enumerate(fake_db):
        if q["id"] == question_id:
            question_index = index
            break

    if question_index == -1:
        raise HTTPException(status_code=404, detail="Question not
found")

    # Create the updated question object
    updated_question = {
        "id": question_id,
        "question": question_update.question,
        "answer": question_update.answer
    }

    # Replace the old dictionary with the new one in the list
    fake_db[question_index] = updated_question

    return updated_question
```

4. Test It:
   - Using `/docs`: Find the new `PUT` endpoint. Try updating question with `id` 1. In the request body, provide a new `question` and `answer`. Execute and observe the `200 OK` response with the updated data.

- Using `curl`:

```
curl -X PUT \
  -H "Content-Type: application/json" \
  -d '{"question": "What is REST?", "answer": "An architectural style
for APIs."}' \
  http://127.0.0.1:8000/questions/1 | jq

# Verify the change
curl http://127.0.0.1:8000/questions/1 | jq
```

Part 4: (Bonus) Implement `PATCH` (Partial Update)

1. The Goal: Create an endpoint that updates *only* the fields provided by the user.
2. Create a `PATCH` Model:
   - For a partial update, all fields should be optional.

```
from typing import Optional # Add Optional to your imports

class QuestionPatch(BaseModel):
    question: Optional[str] = None
    answer: Optional[str] = None
```

3. Add the `PATCH` Endpoint:

```
@app.patch("/questions/{question_id}", response_model=Question)
async def patch_question(question_id: int, question_patch: QuestionPatch):
    question_to_update = None
    for q in fake_db:
        if q["id"] == question_id:
            question_to_update = q
            break

    if question_to_update is None:
        raise HTTPException(status_code=404, detail="Question not found")

    # Update only the fields that were provided in the request
    if question_patch.question is not None:
        question_to_update["question"] = question_patch.question
    if question_patch.answer is not None:
        question_to_update["answer"] = question_patch.answer
```

```
    return question_to_update
```
```

4. Test It:
    - Use `/docs` or `curl` to update *only* the answer of a question, leaving the original question text intact.

### Option B: Build a New IoT Device API

Part 1: Setup

1. Create a New File:
    - In your `Day2_Python_Labs` folder, create a new file named `iot_api.py`.
2. Initial Code:
    - Add the basic imports and app setup:

```python
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from typing import List, Optional

app = FastAPI()
```

Part 2: Define Pydantic Models for a Device

1. The Goal: Model an IoT device with an `id`, `name`, `location`, and `status`.
2. Add the Models:
    - Add some class definitions to `iot_api.py`:

```python
class DeviceBase(BaseModel):
    name: str
    location: str

class DeviceCreate(DeviceBase):
    pass

class DeviceUpdate(BaseModel):
    name: Optional[str] = None
    location: Optional[str] = None
    status: Optional[str] = None # e.g., "online", "offline", "error"

class Device(DeviceBase):
```
```

```
    id: int
    status: str = "offline" # Default status for a new device
```

Part 3: Create an In-Memory Database and All CRUD Endpoints

1. Create the Fake DB:
   - Add a list like this to your file after `app = FastAPI()`:

```
# Simple in-memory database for IoT devices
fake_devices_db = [
    {"id": 1, "name": "Living Room Thermostat", "location": "LR-01",
"status": "online"},
    {"id": 2, "name": "Kitchen Smart Plug", "location": "KT-01",
"status": "offline"},
]
```

2. Implement All Five Endpoints:
   - Using the knowledge from the lectures and previous labs, implement the following five endpoints from scratch. Use the code from Option A as a reference for the logic (finding items, updating, deleting, etc.), but adapt it for your `Device` models and `fake_devices_db`.
   - `POST /devices/`
     - Request Body: `DeviceCreate`
     - Response Model: `Device`
     - Logic: Generate a new `id`, create a new device dictionary (with default status "offline"), add it to the DB, and return it.
   - `GET /devices/`
     - Response Model: `List[Device]`
     - Logic: Return the entire `fake_devices_db`.
   - `GET /devices/{device_id}`
     - Response Model: `Device`
     - Logic: Find the device by `id`. If not found, raise `404`.
   - `DELETE /devices/{device_id}`
     - Response Model: `Device`
     - Logic: Find the device by `id`, remove it from the DB, and return the deleted device. If not found, raise `404`.
   - `PUT /devices/{device_id}`
     - Request Body: `DeviceCreate` (for full replacement)
     - Response Model: `Device`
     - Logic: Find the device by `id`. Replace its `name` and `location` with the data from the request body. Keep the original `id` and `status`. Return

the updated device. If not found, raise `404`.

- (Bonus) `PATCH /devices/{device_id}`
  - Request Body: `DeviceUpdate`
  - Response Model: `Device`
  - Logic: Find the device by `id`. Update *only* the fields (`name`, `location`, `status`) that were provided in the request body. Return the updated device.

Part 4: Run and Test Your New API

1. Run the Server:
   - In your terminal, make sure to run your new file:

     ```
     uvicorn iot_api:app --reload
     ```

2. Test Thoroughly:
   - Open `http://127.0.0.1:8000/docs`.
   - Systematically test every endpoint you created.
     - Create a new device.
     - Get all devices to see your new one.
     - Get the specific device you created.
     - Update it using `PUT`.
     - Partially update it using `PATCH` (if you did the bonus).
     - Delete it.
     - Verify it's gone.
   - Use `curl` for at least one `POST` and one `DELETE` command to practice.